

Department of Computer and Software Engineering – ITU

CE101T: Object Oriented Programming

Course Instructor: Nadir Abbas	Dated: 09/03/2025
Teaching Assistant: Zainab, Nahal, Bilal	Semester: Spring 2025
Session: 2024-2028	Batch: BSCE2024

Assignment 7. Exploring Object-Oriented Relationships – Aggregation, and Composition

Name	Roll number	Obtained Marks/35

Checked on: _____

Signature: _____

Submission:

- Email instructor or TA if there are any questions. You cannot look at others' solutions or use others' solutions, however, you can discuss it with each other. Plagiarism will be dealt with according to the course policy.
- Submission after due time will not be accepted.

In this assignment you have to do following tasks:

Task 1: Ensure that you have installed all three softwares in your personal computer (Github, Cygwin & CLion). Now, accept the assignment posted in classroom (e.g Google, LMS etc) and after accepting, clone the repository to your computer. Make sure you have logged into the github app with your account.

Task 2: Open Cygwin app, Move to your code directory with following command “cd <path_of_folder>”

<path_of_folder> can be automatically populated by dragging the folder and dropping it to the cygwin window.

Run the code through Cygwin, use command “make run”, to get the output of the code

Task 3: Solve the given problems, write code using **CLion** or any other IDE.

Task 4: Keep your code in the respective git cloned folder.

Task 5: Commit and Push the changes through the Github App

Task 5: Write the code in separate files (**as instructed**). Ensure that file names are in lowercase (e,g **main.cpp**).

Task 6: Run ‘**make run**’ to run C++ code

Task 7: Run ‘**make test**’ to test the C++ code

Question 1: Composition - Computer and Processor

Scenario: A **Computer** has a **Processor** that cannot exist without it.

Class Design:

Class Processor

Private Members:

- string brand
- int cores

Public Functions:

- Processor()
- void setBrand(string) & string getBrand()
- void setCores(int) & int getCores()
- bool isHighPerformance() → Returns true if cores > 4

Class Computer

Private Members:

- string model
- Processor processor (Composition)

Public Functions:

- Computer()
- void setModel(string) & string getModel()
- void setProcessor(string, int) // Sets Processor brand and cores
- Processor getProcessor()
- bool isGamingComputer() → Returns true if processor is high performance

Question 2: Aggregation - University and Professors

Scenario: A **University** has **Professors**, but a professor can exist independently of the university.

Class Design:

Class Professor

Private Members:

- string name
- string subject

Public Functions:

- Professor()
- void setName(string) & string getName()
- void setSubject(string) & string getSubject()
- bool teachesSubject(string) → Returns true if professor teaches the given subject

Class University

Private Members:

- string universityName
- Professor* professors[10] (Aggregation using array)
- int professorCount

Public Functions:

- University()
- void setUniversityName(string) & string getUniversityName()
- void addProfessor(Professor*)
- Professor* getProfessor(int)
- int countProfessors() → Returns the number of professors

Question 3: Aggregation - Hospital, Doctors, and Patients

Scenario: A **Hospital** has multiple **Doctors**, and each doctor treats multiple **Patients**.

Class Design:

Class Patient

Private Members:

- string name
- int age

Public Functions:

- Patient()
- void setName(string) & string getName()
- void setAge(int) & int getAge()
- bool isMinor() → Returns true if age < 18

Class Doctor

Private Members:

- string name
- string specialization
- Patient* patients[10] (Aggregation using array)
- int patientCount

Public Functions:

- Doctor()
- void setName(string) & string getName()
- void setSpecialization(string) & string getSpecialization()
- void assignPatient(Patient*)
- Patient* getPatient(int)
- int countPatients() → Returns the number of assigned patients

Class Hospital**Private Members:**

- string hospitalName
- Doctor* doctors[10] (Aggregation using array)
- int doctorCount

Public Functions:

- Hospital()
- void setHospitalName(string) & string getHospitalName()
- void addDoctor(Doctor*)
- Doctor* getDoctor(int)
- int countDoctors() → Returns the number of doctors

Question 4: Composition - Music Album and Songs

Scenario: A **MusicAlbum** consists of multiple **Songs**, and songs cannot exist independently of the album.

Class Design:**Class Song****Private Members:**

- string title
- double duration

Public Functions:

- Song()
- void setTitle(string) & string getTitle()
- void setDuration(double) & double getDuration()
- bool isLongSong() → Returns true if duration > 5.0 minutes

Class MusicAlbum**Private Members:**

- string albumName
- Song songs[10] (Composition using array)
- int songCount

Public Functions:

- MusicAlbum()
- void setAlbumName(string) & string getAlbumName()
- void addSong(string, double)
- Song getSong(int)
- int countSongs() → Returns the number of songs in the album

Question 5: Composition - Restaurant, Menu, and Dish

Scenario: A **Restaurant** has a **Menu**, and each **Menu** contains multiple **Dishes** that cannot exist independently of the menu.

Class Design:

Class Dish

Private Members:

- string name
- double price

Public Functions:

- Dish()
- void setName(string) & string getName()
- void setPrice(double) & double getPrice()
- bool isExpensiveDish() → Returns true if price > 20

Class Menu

Private Members:

- Dish dishes[10] (Composition using array)
- int dishCount

Public Functions:

- Menu()
- void addDish(string, double)
- Dish getDish(int)
- int countDishes() → Returns the number of dishes in the menu

Class Restaurant

Private Members:

- string restaurantName
- Menu menu (Composition)

Public Functions:

- Restaurant()
- void setRestaurantName(string) & string getRestaurantName()
- void setMenu(string, double) // Sets a dish inside menu
- Menu getMenu()
- bool hasExpensiveDishes() → Returns true if any dish is expensive

Question 6: UML Diagram - Restaurant, Menu, and Dish (Composition) & Hospital, Doctors, and Patients (Aggregation)

- Draw the UML diagram for the **Restaurant, Menu, and Dish** (Composition) example.
- Draw the UML diagram for the **Hospital, Doctors, and Patients** (Aggregation) example.

UML Diagram Requirements:

- Show class names, attributes, and functions.
- Use correct relationships.
- Make UML diagrams in a single image.
- Submit as an image file (e.g., UML_Diagram.png).

Assessment Rubric for Assignment

Performance metric	CL O	Able to complete the task over 80% (4-5)	Able to complete the task 50-80% (2-3)	Able to complete the task below 50% (0-1)	Marks
1. Realization of experiment	3	Executes without errors excellent user prompts, good use of symbols, spacing in output. Through testing has been completed.	Executes without errors, user prompts are understandable, minimum use of symbols or spacing in output. Some testing has been completed.	Does not execute due to syntax errors, runtime errors, user prompts are misleading or non-existent. No testing has been completed.	
2. Conducting experiment	2	Able to make changes and answered all questions.	Partially able to make changes and few incorrect answers.	Unable to make changes and answer all questions.	
3. Computer use	4	Document submission timely.	Document submission late.	Document submission not done.	
4. Teamwork	4	Actively engages and cooperates with other group member(s) in effective manner.	Cooperates with other group member(s) in a reasonable manner but conduct can be improved.	Distracts or discourages other group members from conducting the experiment	
5. Laboratory safety and disciplinary rules	2	Code comments are added and does help the reader to understand the code.	Code comments are added and does not help the reader to understand the code.	Code comments are not added.	
6. Data collection	2	Excellent use of white space, creatively organized work, excellent use of variables and constants, correct identifiers for constants, No line-wrap.	Includes name, and assignment, white space makes the program fairly easy to read. Title, organized work, good use of variables.	Poor use of white space (indentation, blank lines) making code hard to read, disorganized and messy.	
7. Data analysis	3	Solution is efficient, easy to understand, and maintain.	A logical solution that is easy to follow but it is not the most efficient.	A difficult and inefficient solution.	
Total (out of 35):					